

HDOC Day-30

Write a C program to perform the following menu-driven operations on a given array:

1. Sort the elements using Bubble Sort algorithm and perform a Binary search.
2. Perform Insertion and Deletion in a given valid position.
3. Rotate the array elements to the right by k positions.
4. Exit

Algorithm

1. Bubble Sort and Binary Search

1. Start.
2. Input the array elements.
3. Compare adjacent elements.
4. Swap them if the first element is greater than the second.
5. Repeat until the array is sorted.
6. Set low = 0 and high = n-1.
7. Find mid = (low + high)/2.
8. Compare the search element with arr[mid].
9. If equal, display its position.
10. If smaller, search the left half.
11. If greater, search the right half.
12. Repeat until the element is found or low > high.
13. Stop.

2. Insertion

1. Start.
2. Input the position and value.
3. Check whether the position is valid.
4. Shift elements one position to the right.
5. Insert the new element.

6. Increase the array size by 1.
7. Display the array.
8. Stop.

3. Deletion

1. Start.
2. Input the position to delete.
3. Check whether the position is valid.
4. Shift all elements after that position one position to the left.
5. Decrease the array size by 1.
6. Display the array.
7. Stop.

4. Right Rotation by k

1. Start.
2. Input k.
3. Calculate $k = k \% n$.
4. Store the last element temporarily.
5. Shift all elements one position to the right.
6. Place the last element at the first position.
7. Repeat the process k times.
8. Display the rotated array.
9. Stop.

● C Program:

- `#include <stdio.h>`
- `#define MAX 100`
-
- `// Function to display array`
- `void display(int arr[], int n)`
- `{`
- `int i;`
-

- printf("Array: ");
- for (i = 0; i < n; i++)
- {
- printf("%d ", arr[i]);
- }
- printf("\n");
- }
-
- // Bubble Sort
- void bubbleSort(int arr[], int n)
- {
- int i, j, temp;
-
- for (i = 0; i < n - 1; i++)
- {
- for (j = 0; j < n - i - 1; j++)
- {
- if (arr[j] > arr[j + 1])
- {
- temp = arr[j];
- arr[j] = arr[j + 1];
- arr[j + 1] = temp;
- }
- }
- }
- }
-
- // Binary Search
- int binarySearch(int arr[], int n, int key)
- {
- int low = 0, high = n - 1, mid;
-
- while (low <= high)
- {
- mid = (low + high) / 2;
-
- if (arr[mid] == key)
- return mid;
-

- else if (arr[mid] < key)
- low = mid + 1;
-
- else
- high = mid - 1;
- }
-
- return -1;
- }
-
- // Insertion
- void insertElement(int arr[], int *n, int position, int value)
- {
- int i;
-
- for (i = *n; i >= position; i--)
- {
- arr[i] = arr[i - 1];
- }
-
- arr[position - 1] = value;
- (*n)++;
-
- printf("Element inserted successfully.\n");
- }
-
- // Deletion
- void deleteElement(int arr[], int *n, int position)
- {
- int i;
-
- for (i = position - 1; i < *n - 1; i++)
- {
- arr[i] = arr[i + 1];
- }
-
- (*n)--;
-
- printf("Element deleted successfully.\n");

- }
-
- // Right Rotation
- void rotateRight(int arr[], int n, int k)
- {
- int i, j, temp;
-
- k = k % n;
-
- for (i = 0; i < k; i++)
- {
- temp = arr[n - 1];
-
- for (j = n - 1; j > 0; j--)
- {
- arr[j] = arr[j - 1];
- }
-
- arr[0] = temp;
- }
- }
-
- int main()
- {
- int arr[MAX], n, i;
- int choice, position, value, key, result, k;
- int subChoice;
-
- printf("Enter number of elements: ");
- scanf("%d", &n);
-
- printf("Enter %d elements:\n", n);
- for (i = 0; i < n; i++)
- {
- scanf("%d", &arr[i]);
- }
-
- do
- {

- printf("\n===== MENU =====\n");
- printf("1. Bubble Sort and Binary Search\n");
- printf("2. Insertion and Deletion\n");
- printf("3. Rotate Array Right by k Positions\n");
- printf("4. Exit\n");
- printf("=====\n");
-
- printf("Enter your choice: ");
- scanf("%d", &choice);
-
- switch (choice)
- {
- case 1:
- // Sort array
- bubbleSort(arr, n);
-
- printf("\nArray after Bubble Sort:\n");
- display(arr, n);
-
- // Binary Search
- printf("Enter element to search: ");
- scanf("%d", &key);
-
- result = binarySearch(arr, n, key);
-
- if (result != -1)
- printf("Element %d found at position %d.\n",
- key, result + 1);
- else
- printf("Element %d not found in the array.\n", key);
-
- break;
-
- case 2:
- printf("\n1. Insert an element\n");
- printf("2. Delete an element\n");
- printf("Enter your choice: ");
- scanf("%d", &subChoice);
-

- if (subChoice == 1)
- {
- if (n >= MAX)
- {
- printf("Array is full. Insertion not possible.\n");
- break;
- }
-
- printf("Enter position (1 to %d): ", n + 1);
- scanf("%d", &position);
-
- if (position < 1 || position > n + 1)
- {
- printf("Invalid position.\n");
- break;
- }
-
- printf("Enter value to insert: ");
- scanf("%d", &value);
-
- insertElement(arr, &n, position, value);
- display(arr, n);
- }
- else if (subChoice == 2)
- {
- if (n == 0)
- {
- printf("Array is empty. Deletion not possible.\n");
- break;
- }
-
- printf("Enter position (1 to %d): ", n);
- scanf("%d", &position);
-
- if (position < 1 || position > n)
- {
- printf("Invalid position.\n");
- break;
- }

-
- deleteElement(arr, &n, position);
- display(arr, n);
- }
- else
- {
- printf("Invalid choice.\n");
- }
-
- break;
-
- case 3:
- if (n == 0)
- {
- printf("Array is empty.\n");
- break;
- }
-
- printf("Enter value of k: ");
- scanf("%d", &k);
-
- if (k < 0)
- {
- printf("k must be non-negative.\n");
- break;
- }
-
- rotateRight(arr, n, k);
-
- printf("Array after right rotation:\n");
- display(arr, n);
-
- break;
-
- case 4:
- printf("Program terminated successfully.\n");
- break;
-
- default:

- printf("Invalid choice. Please try again.\n");
- }
-
- } while (choice != 4);
-
- return 0;
- }

Test Case Table:

Test Case	Initial array	Choice	Additional Input	Expected Output
1	5 2 8 1 4	1	Search : 8	1 2 4 5 8
2	5 2 8 1 4	1	Search : 7	Element 7 not found in the array
3	5 2 8 1 4	2	Sub choice : 1, position:3,	5 2 10 8 1 4
4	5 2 10 8 1	2	Sub choice: 2, Position: 4	5 2 10 1
5	1 2 3 4 5	3	K = 2	4 5 1 2 3
6	1 2 3 4 5	3	K = 7	2 3 4 5 1
7	1 2 3 4 5	2	Sub choice: 1 , Position = 0	Invalid position
8	Any array	4	-	Program terminated successfully

Compilation and Execution:

Test Case 1:

```

(kali@kali)-[~]
└─$ ./a.out
Enter number of elements: 5
Enter 5 elements: 5 2 8 1 4

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
=====
Enter your choice: 1

Array after Bubble Sort:
Array: 1 2 4 5 8
Enter element to search: 8
Element 8 found at position 5.

```

Test Case 2:

```

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
=====
Enter your choice: 1

Array after Bubble Sort:
Array: 1 2 4 5 8
Enter element to search: 7
Element 7 not found in the array.

```

Test Case 3:


```
Swap file ".experiment.c.swp" already
(kali@kali)-[~]E)dit anyway, (R)ec
$ ./a.out ".experiment.c.swp" already
Enter number of elements: 5
Enter 5 elements:
5 2 10 8 1

===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
=====
Enter your choice: 2

1. Insert an element
2. Delete an element
Enter your choice: 2
Enter position (1 to 5): 4
Element deleted successfully.
Array: 5 2 10 1
```

Test Case 5:

```
(kali㉿kali)-[~]  
└─$ ./a.out  
Enter number of elements: 5  
Enter 5 elements:  
1 2 3 4 5  
  
===== MENU =====  
1. Bubble Sort and Binary Search  
2. Insertion and Deletion  
3. Rotate Array Right by k Positions  
4. Exit  
=====
```

Enter your choice: 3
Enter value of k: 2
Array after right rotation:
Array: 4 5 1 2 3

Test Case 6:

```
===== MENU =====  
1. Bubble Sort and Binary Search  
2. Insertion and Deletion  
3. Rotate Array Right by k Positions  
4. Exit  
=====
```

Enter your choice: 3
Enter value of k: 7
Array after right rotation:
Array: 2 3 4 5 1

Test Case 7:

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
=====
Enter your choice: 2

1. Insert an element
2. Delete an element
Enter your choice: 1
Enter position (1 to 6): 0
Invalid position.
```

Test Case 8:

```
===== MENU =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
=====
Enter your choice: 4
Program terminated successfully.
```